

Clase 3: Data Warehouse y ETL

Módulo 7: Data Pipelines y plataformas de visualización

Claudio Torres Fonseca

mail: `claudio.torresf@usm.cl`

Universidad de Santiago de Chile

Diplomado en Data Engineer
2026

Contenido

- 1 Introducción
- 2 Almacenamiento de datos
- 3 Flujos de datos
- 4 Caso de estudio
 - Arquitectura GlobalShop 360
 - Retail Omnicanal

Contenido

- 1 Introducción
- 2 Almacenamiento de datos
- 3 Flujos de datos
- 4 Caso de estudio

ETL vs ELT

Característica	ETL (Tradicional)	ELT (Moderno)
<i>Orden</i>	Transforma antes de cargar	Limpia y modela después de cargar
<i>Lugar de Transformación</i>	Servidor de procesamiento intermedio (a menudo on-premise)	Dentro del destino (Data Warehouse o Data Lake)
<i>Tipo de Datos</i>	Carga solo datos limpios, agregados y modelados.	Carga datos crudos y sin procesar.
<i>Flexibilidad</i>	Menor, ya que los cambios en la lógica de T son costosos.	Mayor, ya que los datos crudos están siempre disponibles . para nuevos análisis.

¿Por qué el patrón ELT ha ganado popularidad?

- El patrón ELT siempre fue una opción teórica, pero su implementación práctica se volvió viable y atractiva gracias a la convergencia de tres factores clave de la nube:
 - 1 Almacenamiento Barato y Escalable.
 - 2 Capacidad de Cómputo Elástica y a Demanda.
 - 3 Flexibilidad y Agilidad.

Bases de datos

- ¿Cuál es la diferencia entre una base de datos de Transacciones y una base de datos Analítica?

OLTP vs OLAP

OLTP

- 1 Procesamiento de Transacciones en Línea.
- 2 Permite administrar y procesar transacciones en tiempo real.
- 3 Utiliza datos transaccionales y en tiempo real de un solo origen.
- 4 gigabytes (GB).

OLAP

- 1 Procesamiento Analítico en Línea.
- 2 Permite analizar grandes volúmenes de datos para respaldar la toma de decisiones.
- 3 Utiliza datos históricos y agregados de varios orígenes.
- 4 terabytes (TB) y petabytes (PB).

Archivos

- ¿Cuál es la diferencia entre Json, Avro y Parquet?

Json

Serialización basada en filas

- JavaScript Object Notation
- Formato ligero y legible por humanos y máquinas que se usa para almacenar e intercambiar datos estructurados.
- Estándar para el intercambio de datos a través de API.
- Se ha hecho popular con el auge de bases de datos de documentos, como MongoDB.
- Bases de datos como Snowflake, BigQuery y SQL Server también ofrecen un amplio soporte nativo, lo que facilita el intercambio de datos entre aplicaciones, API y sistemas de bases de datos.

Json

Serialización basada en filas

Json vacío

```
{ }
```

Json simple

```
{  
  "StringProperty": "StrValue",  
  "NumberProperty": 10,  
  "FloatProperty": 20.13,  
  "BooleanProperty": true,  
  "EmptyProperty": null,  
}
```

Objeto con campos

```
{  
  "object": {  
    "Name": "Neste  Object"  
  },  
  "array": [  
    10,  
    20,  
    40  
  ]  
}
```

JSONL

Serialización basada en filas

JSONL

- JSON Lines
- Almacena una secuencia de objetos JSON, delimitados por saltos de línea
- Es un formato extremadamente útil para almacenar datos inmediatamente después de su ingesta desde API o aplicaciones

Serialización

Serialización basada en filas

JSONL - Ejemplo

```
{"nombre": "Alice", "edad": 30, "ciudad": "Nueva York"}  
{"nombre": "Bob", "edad": 25, "ciudad": "Londres"}  
{"nombre": "Charlie", "edad": 35, "ciudad": "Tokio"}
```

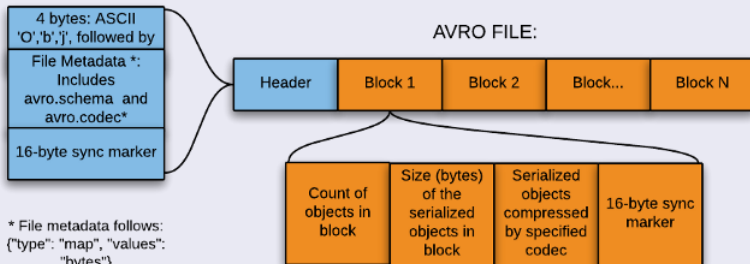
Apache Avro

Serialización basada en filas

- Formato de datos orientado a filas
- Codifica los datos en formato binario, con metadatos de esquema especificados en JSON
- Popular en el ecosistema Hadoop y también es compatible con diversas herramientas de datos en la nube
- Doc:
 - <https://avro.apache.org/docs/1.12.0/specification/>
 - <https://avro.apache.org/docs/1.12.0/getting-started-java/>

Apache Avro

Serialización basada en filas



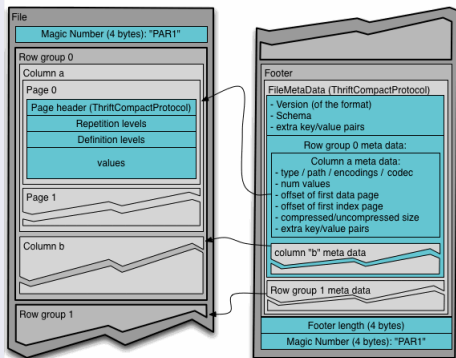
Apache Parquet

Serialización basada en columnas

- Almacena datos en formato columnar y está diseñado para lograr un excelente rendimiento de lectura y escritura en un entorno de data lake
- Los datos codificados en Parquet incorporan información de esquema y admiten datos anidados de forma nativa, a diferencia de CSV
- Se utiliza con diversos algoritmos de compresión

Apache Parquet

Serialización basada en columnas



Formatos

Links

- https://github.com/ctorresf/developer-guides/blob/main/python_data_engineering/format_benchmarks_with_pandas.py
- https://github.com/ctorresf/developer-guides/blob/main/apache_beam/vscode-devcontainer-basic-beam-examples/src/json_to_formats.py

Contenido

- 1 Introducción
- 2 Almacenamiento de datos**
- 3 Flujos de datos
- 4 Caso de estudio

Arquitecturas Heredadas

Sistema Mainframe (unidad central)

- Es una computadora de alto rendimiento
- Diseñada para procesar volúmenes masivos de datos y miles de millones de transacciones simultáneas de forma segura y fiable.
- Utilizados por grandes organizaciones (bancos, aerolíneas, gobiernos), estos equipos gestionan aplicaciones críticas de misión, destacando por su enorme capacidad de E/S (entrada/salida), escalabilidad y disponibilidad ininterrumpida.

Arquitecturas Heredadas

Sistema on-premise (on-prem)

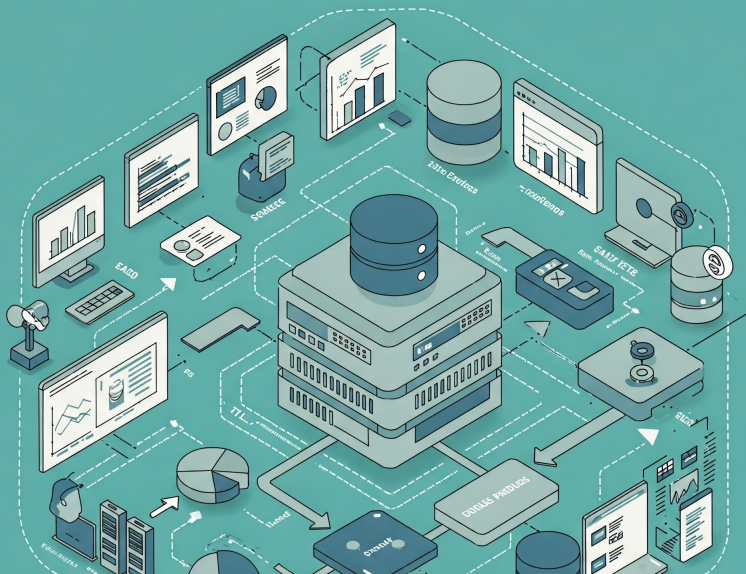
- Es software e infraestructura de TI instalados y operados dentro del centro de datos o instalaciones físicas de una empresa, en lugar de en servidores remotos o en la nube.
- Ofrece control total, seguridad personalizada y alta personalización, siendo ideal para empresas con normativas estrictas.

Arquitecturas Heredadas

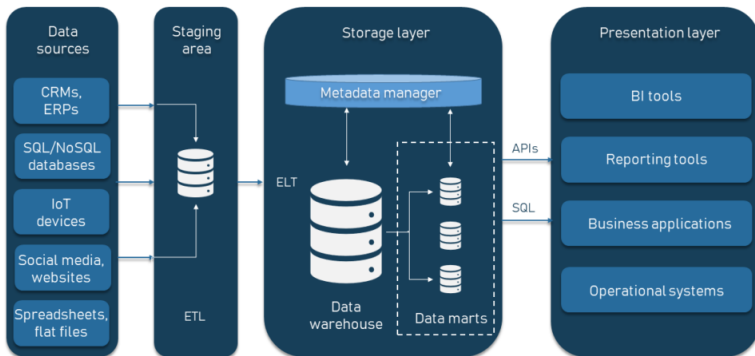
Data Warehouse on-premise

- Es un almacén de datos centralizado, integrado y no volátil
- Instalado y ejecutado en los servidores y el hardware físico propiedad de la propia empresa, dentro de sus propias instalaciones físicas.
- Ofrece control total, seguridad directa y soberanía sobre los datos, sin depender de internet o proveedores externos.

DATA WAREHOUSE



ENTERPRISE DATA WAREHOUSE COMPONENTS



Arquitecturas Heredadas

Limitaciones

- Escalabilidad vertical costosa
- Silos de datos
- Procesos ETL rígidos
- Alta latencia (solo batch nocturno)

Data Lake

Definición

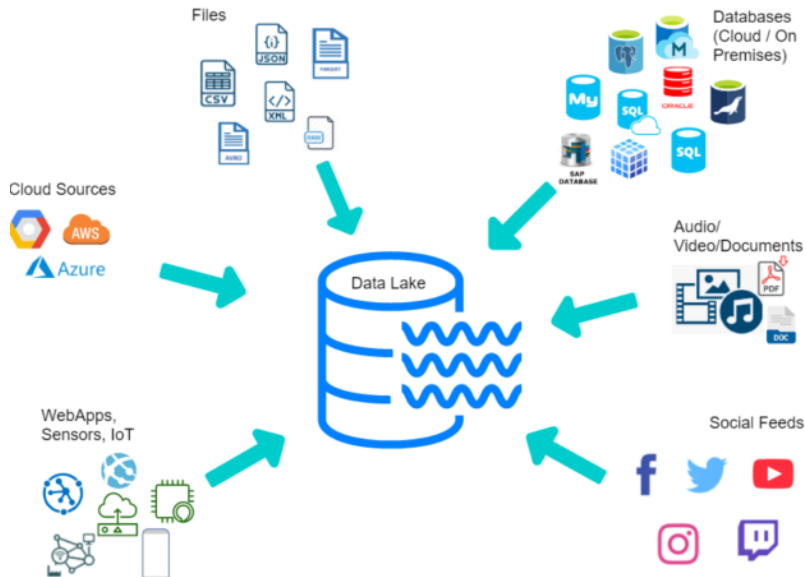
- Repositorio centralizado que permite almacenar todos los datos (estructurados y no estructurados) a cualquier escala.

Schema-on-read

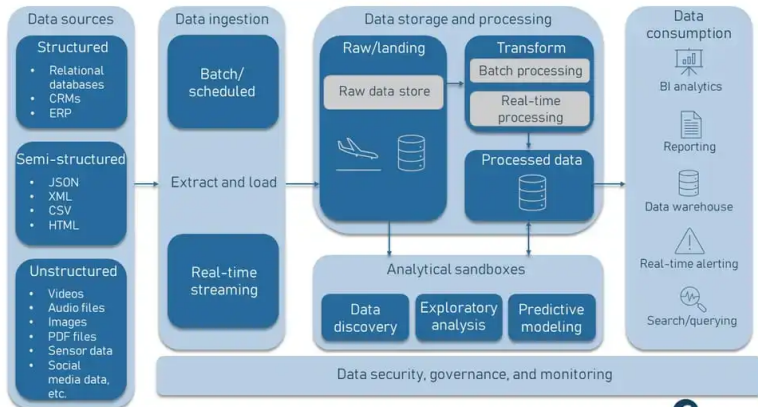
- El esquema se define al consultar

Schema-on-write

- El esquema se define al escribir (esquema rígido)
- Usado en Datawarehouse



DATA LAKE ARCHITECTURE



DATA LAKE

vs

DATA WAREHOUSE



Raw

Data Lakes contain unstructured, semi structured and structured data with minimal processing. It can be used to contain unconventional data such as log and sensor data

Large

Data Lakes contain vast amounts of data in the order of petabytes. Since the data can be in any form or size, large amounts of unstructured data can be stored indefinitely and can be transformed when in use only

Undefined

Data in data lakes can be used for a wide variety of applications, such as Machine Learning, Streaming analytics, and AI



Refined

Data Warehouses contain highly structured data that is cleaned, pre-processed and refined. This data is stored for very specific use cases such as BI.

Smaller

Data Warehouses contain less data in the order of terabytes. In order to maintain data cleanliness and health of the warehouse, Data must be processed before ingestion and periodic purging of data is necessary

Relational

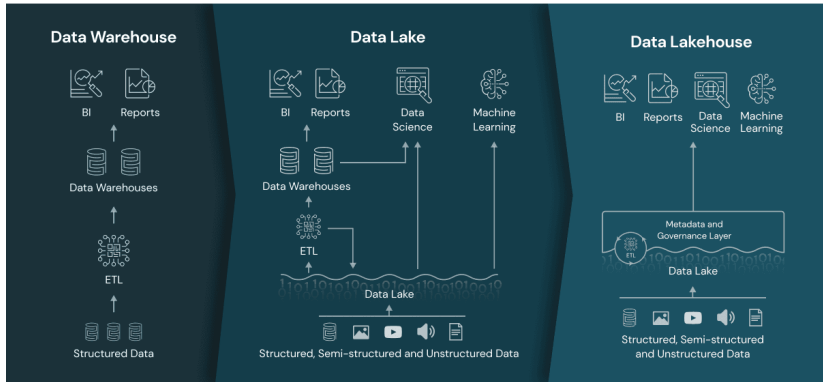
Data Warehouses contain historic and relational data, such as transaction systems, operations etc



Data Lakehouse

Concepto

- Lo mejor de ambos mundos.
- La estructura y gestión de datos del Warehouse sobre el almacenamiento barato y flexible del Data Lake.
- **Tecnologías:** Apache Iceberg, Delta Lake.



Contenido

- 1 Introducción
- 2 Almacenamiento de datos
- 3 Flujos de datos**
- 4 Caso de estudio

Arquitecturas Lambda y Kappa

Problema

- Surgen para resolver un problema fundamental en la ingeniería de datos: La tiranía del compromiso entre Velocidad y Consistencia.
- Antes de estas arquitecturas, las empresas tenían que elegir:
 - o procesaban datos rápido (pero con errores o datos incompletos)
 - o procesaban datos exactos (pero horas o días después).

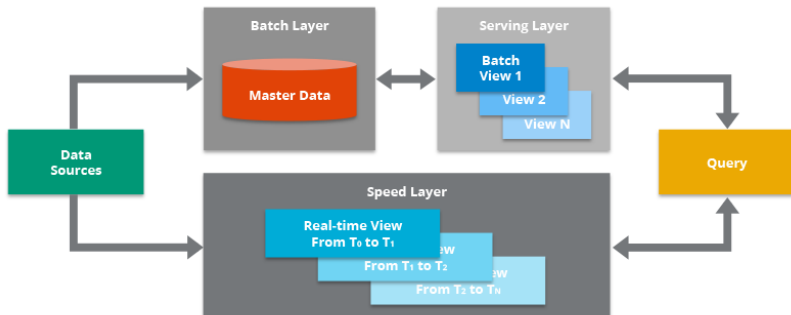
Arquitectura Lambda (El puente)

Concepto

- Dividir el flujo en dos caminos para resolver el problema de la latencia sin perder exactitud histórica.

Capas

- **Batch:** Datos históricos, alta precisión, latencia alta (ej. Hadoop/Spark).
- **Speed:** Datos en tiempo real, latencia mínima, precisión aproximada (ej. Storm/Flink).
- **Serving:** Une ambas vistas para el usuario final.



Arquitectura Kappa (La simplificación)

Concepto

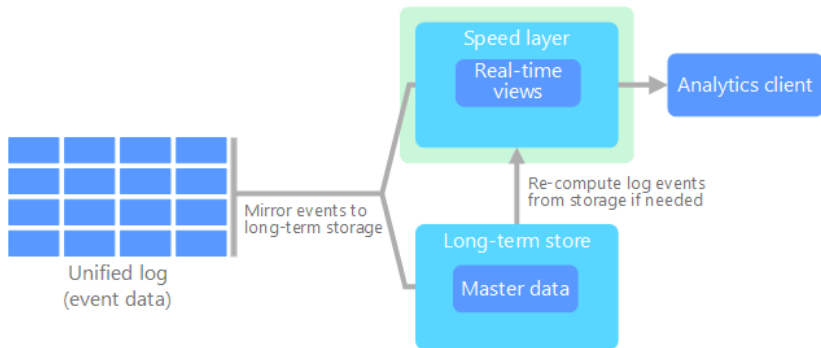
- **Todo es un flujo (stream).**
- Eliminamos la capa de batch.

Mecanismo

- Se utiliza un log de eventos (ej. Kafka) como fuente de verdad única.

Ventaja

- Un solo código base para mantener. Si se necesita reprocesar el pasado, se vuelven a leer los eventos desde el inicio del log.



Data Fabric

Definición

- Una capa de gestión de datos unificada que utiliza metadatos para conectar fuentes dispares.

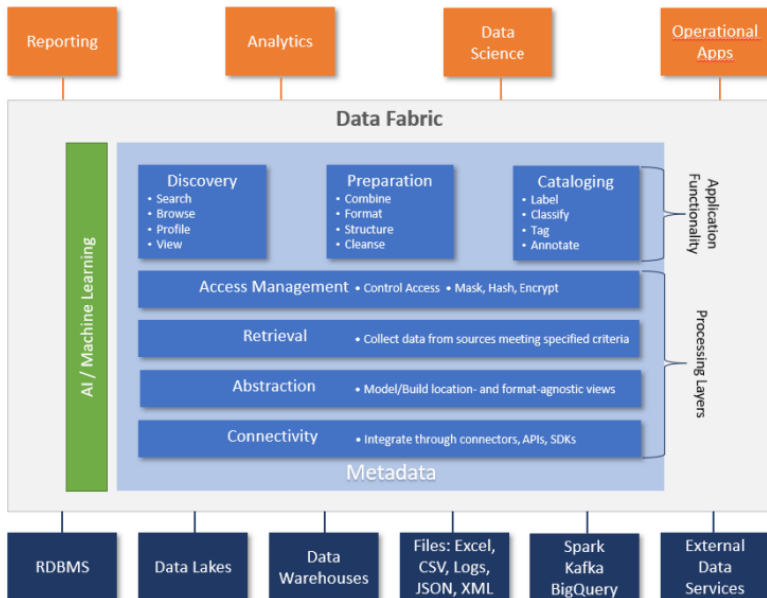
Foco

- Automatización e integración tecnológica. Ideal para empresas con nubes híbridas.

Componente clave

- El Catálogo de Datos activo.

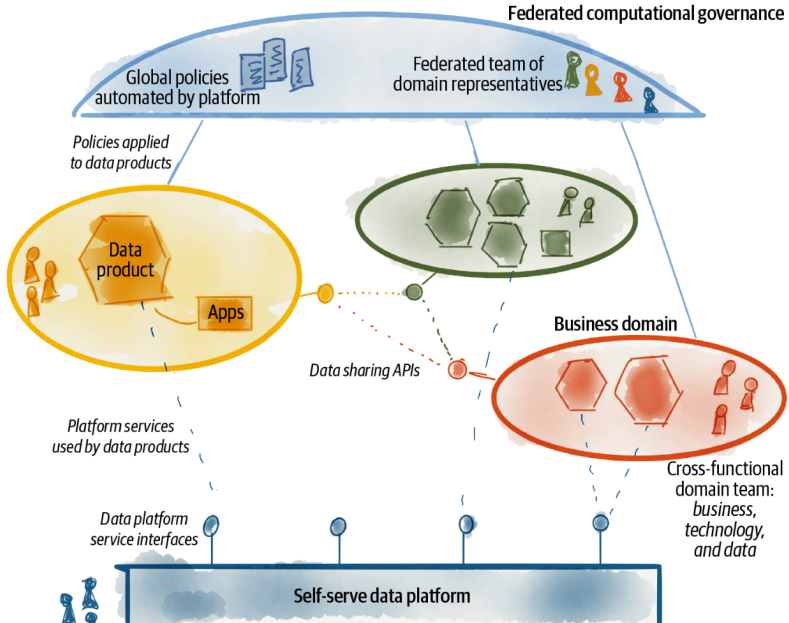
Data Consumption

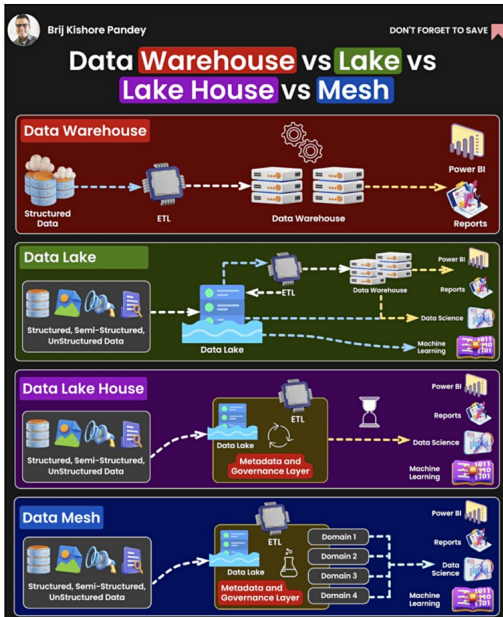


Data Mesh

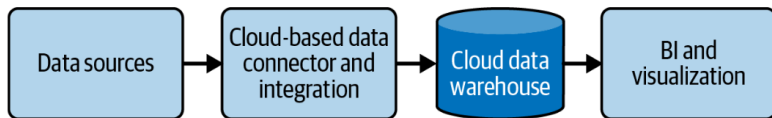
Pilares

- Propiedad de datos orientada a dominios (Por ejemplo: Marketing es dueño de sus datos).
- Datos como Producto (Data as a Product).
- Plataforma de datos de autoservicio.
- Gobernanza federada computacional.





El Modern Data Stack (MDS)



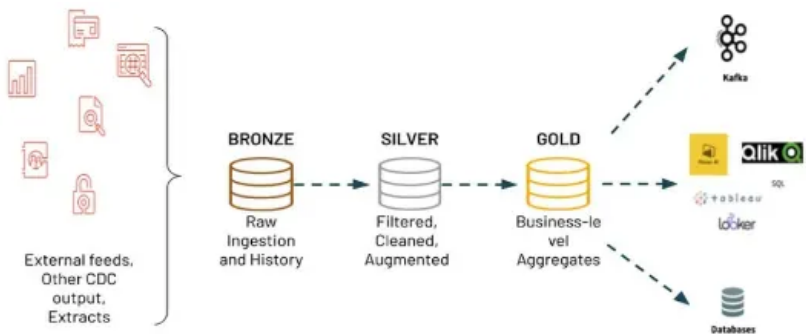
El Modern Data Stack (MDS)

2026

Componentes

- Ingesta: Airbyte / dlt.
- Almacenamiento: Snowflake / Bigquery.
- Transformación: dbt (SQL-first).
- BI/Serving: Lightdash / Evidence.dev.
- Orquestación: Dagster / Airflow.

Medallion Architecture



(Image adopted from Databricks, Inc.)

Medallion Architecture

Definición

- Es un diseño de datos en capas (Bronce, Plata, Oro) utilizado en data lakehouses para estructurar, limpiar y enriquecer datos.
- Popularizada por Databricks, su objetivo es mejorar la calidad y el valor de los datos progresivamente, desde los datos en bruto hasta los listos para análisis.

Medallion Architecture

Capa Bronze (Raw / Cruda)

- Es el punto de entrada. Aquí volcamos los datos exactamente como vienen de la fuente (APIs, bases de datos, logs).
- **Estado:** Datos "sucios", con duplicados, errores de formato o valores nulos.
- **Regla de Oro:** Inmutabilidad. Nunca se borra nada aquí. Si algo sale mal en las capas siguientes, siempre puedes volver al Bronze para reprocesar.
- **Formato:** Generalmente Parquet o Avro para mantener la eficiencia.

Medallion Architecture

Capa Silver (Cleansed / Limpia)

- Aquí ocurre la primera gran transformación.
- Los datos se limpian, se normalizan y se estructuran en tablas.
- **Acciones:** Eliminación de duplicados, corrección de tipos de datos (ej. convertir strings a fechas), y filtrado de registros corruptos.
- **Relaciones:** Se empiezan a unir tablas (Joins). Por ejemplo, unes la tabla de Ventas con la de Clientes.
- **Uso:** Es la capa favorita de los Data Scientists para entrenar modelos de Machine Learning.

Medallion Architecture

Capa Gold (Curated / De Negocio)

- Es la capa final, lista para el consumo de los usuarios de negocio y herramientas de BI (PowerBI, Tableau).
- **Estado:** Datos altamente agregados y optimizados.
- **Estructura:** aplicamos el Modelo Dimensional (Tablas de Hechos y Dimensiones) del Datawarehouse.
- **Ejemplo:** En lugar de ver cada clic individual, la capa Gold tiene una tabla de Ventas_Mensuales_Por_Pais.

Medallion Architecture en AWS



Contenido

- 1 Introducción
- 2 Almacenamiento de datos
- 3 Flujos de datos
- 4 Caso de estudio
 - Arquitectura GlobalShop 360
 - Retail Omnicanal

Contenido

- 1 Introducción
- 2 Almacenamiento de datos
- 3 Flujos de datos
- 4 Caso de estudio
 - Arquitectura GlobalShop 360
 - Retail Omnicanal

Desafío Práctico: Arquitectura GlobalShop 360

Contexto

- GlobalShop es un gigante del e-commerce que opera en 15 países. Su arquitectura actual de "base de datos única" ha colapsado. La dirección exige una nueva plataforma de datos que resuelva dos necesidades contradictorias:
 - **Detección de Fraude (Real-time):** Analizar transacciones y clics para bloquear estafas en menos de 2 segundos.
 - **Reporte de Ventas Anual (Batch):** Consolidar 500 millones de registros históricos para auditoría legal y cierre contable.

Desafío Práctico: Arquitectura GlobalShop 360

Actividad

Utilizando Excalidraw (<https://excalidraw.com/>) o Lucidchart (<https://www.lucidchart.com/>), debes diseñar una arquitectura de datos integral que cubra todo el Data Engineering Lifecycle. Tu diseño debe responder a:

- A. Selección de Patrón de Procesamiento
 - Lambda
 - Kappa
- B. Almacenamiento
 - Data Lake
 - Data Warehouse

Contenido

- 1 Introducción
- 2 Almacenamiento de datos
- 3 Flujos de datos
- 4 Caso de estudio
 - Arquitectura GlobalShop 360
 - Retail Omnicanal

Retail Omnicanal (Patrón ETL-P con PSA)

Escenario:

- Procesamiento de ventas históricas. Es crítico mantener una copia exacta de lo recibido (PSA) para auditorías legales antes de transformar los montos a la moneda del Data Warehouse.

Retail Omnicanal (Patrón ETL-P con PSA)

Definición de Esquemas y Flujo de Datos

Esquema de Entrada (Raw - CSV)

Es la representación fiel de la fuente. No hay tipos de datos complejos todavía.

- **timestamp (String):** ISO 8601, con Fecha y Hora en UTC (YYYY-MM-DDThh:mm:ssZ). Ejemplo:
2026-04-27T14:30:00Z
- **store_id (String):** Identificador de la sucursal.
- **sku (String):** Código de producto.
- **amount_clp (String):** Monto en pesos chilenos (podría venir con errores o nulos).

Retail Omnicanal (Patrón ETL-P con PSA)

Definición de Esquemas y Flujo de Datos

Esquema PSA (Bronze - Parquet)

Es una copia exacta pero en un formato eficiente para almacenamiento masivo.

- Se mantienen las columnas originales pero se añade **ingestion_timestamp** (para auditoría) y **file_source**.

Retail Omnicanal (Patrón ETL-P con PSA)

Definición de Esquemas y Flujo de Datos

Esquema de Salida (Gold/DWS - Relacional)

Datos listos para el negocio.

- **sale_date (Date).**
- **product_key (Integer):** ID transformado para el Data Warehouse.
- **amount_usd (Float):** Monto normalizado.
- **tax_amount (Float):** Cálculo derivado.

Retail Omnicanal (Patrón ETL-P con PSA)

Transformaciones y Medios de Almacenamiento

El pipeline se divide en dos fases claras conectadas por el almacenamiento persistente:

- **Fase 1 (Ingesta):** Lectura del CSV → Validación básica → Escritura en Google Cloud Storage (GCS) en formato Parquet.
- **Fase 2 (Enriquecimiento):** Lectura de Parquet → Búsqueda de precios de moneda (Side Input) → Conversión → Carga en el Data Warehouse (DWS) (ej. BigQuery o PostgreSQL).

Retail Omnicanal (Patrón ETL-P con PSA)

Análisis

Paso 1: Esquema (El Contrato)

- En este caso, el desafío del esquema es el desacoplamiento.
- La PSA debe ser capaz de absorber cambios en la fuente sin romper el DWS.
- Si el retail añade una columna de 'descuento' mañana, el pipeline de la PSA la guarda automáticamente (esquema evolutivo), pero el DWS solo se actualiza cuando el negocio lo requiera.

Retail Omnicanal (Patrón ETL-P con PSA)

Análisis

Paso 2: DAG (Grafo de Ejecución)

El DAG aquí no es lineal, es un DAG con persistencia.

- **Rama de Respaldo:** El primer nodo del DAG escribe directamente en el PSA. Es una operación de 'Escribir y Olvidar' (Fire and Forget) para asegurar la copia legal.
- **Rama de Valor:** Solo después de que la persistencia es confirmada, se inicia la lógica de negocio (conversión de moneda). Esto garantiza que si la transformación falla, no perdemos la materia prima.

Retail Omnicanal (Patrón ETL-P con PSA)

Análisis

Paso 3: Manejo de Errores (Resiliencia)

Utilizamos el patrón de **Dead-Letter Table** dentro de la PSA.

- Si una fila no tiene amount, Apache Beam usa un TaggedOutput para enviarla a una carpeta **/errors/year=2026/month=03/**.
- Esto permite que el pipeline principal siga corriendo (Disponibilidad) mientras el equipo de TI investiga por qué la sucursal X está enviando datos nulos.

Retail Omnicanal (Patrón ETL-P con PSA)

Análisis

Paso 4: Sinking Strategy (El Destino)

- **PSA (Parquet):** Se elige por ser inmutable y particionable. Podemos particionar por **/fecha/sucursal/**, lo que permite que una auditoría legal de hace 2 años se ejecute en segundos leyendo solo los archivos necesarios.
- **DWS (Tablas):** Aquí el almacenamiento es optimizado para lectura (índices, particiones por fecha de venta).

Retail Omnicanal (Patrón ETL-P con PSA)

Análisis

Paso 5: Las 4 Preguntas de Beam

- **What (¿Qué?):** Estamos calculando el valor financiero global del retail. Se transforma moneda y se limpian nulos.
- **Where (¿Dónde?):** Tiempo de procesamiento Batch. Se analiza el día anterior completo (Ventana global).
- **When (¿Cuándo?):** El disparo del resultado ocurre cuando se lee el End of File (EOF) del lote diario.
- **How (¿Cómo?):** Los resultados son acumulativos en la PSA (siempre guardamos todo) pero pueden ser de reemplazo en el DWS (sobrescribir el reporte diario si se detectan errores y se re-procesa).

Retail Omnicanal (Patrón ETL-P con PSA)

Análisis

Conclusión

- El uso de la PSA en este caso actúa como un 'seguro de vida'.
- Si el algoritmo de conversión de moneda tuviera un error, no tenemos que pedirle los datos de nuevo a las tiendas (lo cual sería costoso o imposible); simplemente reiniciamos el pipeline desde la PSA con el código corregido.

Retail Omnicanal (Patrón ETL-P con PSA)

Solución

- https://github.com/ctorresf/developer-guides/blob/main/apache_beam/vscode-devcontainer-beam-case-study-rabbitmq/src/psa_ingestion.py
- https://github.com/ctorresf/developer-guides/blob/main/apache_beam/vscode-devcontainer-beam-case-study-rabbitmq/src/dws_currency_conversion.py

Retail Omnicanal (Patrón ETL-P con PSA)

Análisis del Diseño

- **Independencia de Fallos:** Si el tipo de cambio (Exchange Rate) está mal configurado, el segundo script fallará, pero los datos originales ya están seguros en la PSA. El ingeniero simplemente corrige el segundo script y lo vuelve a ejecutar apuntando a los archivos Parquet.
- **Uso de Memoria (VSA):** En el segundo script, la transformación ocurre en una Volatile Staging Area (el contexto de ejecución de los workers de Beam). Los datos erróneos se separan mediante TaggedOutput, permitiendo que el flujo 'limpio' continúe hacia el Data Warehouse.

Retail Omnicanal (Patrón ETL-P con PSA)

Análisis del Diseño

- **Auditabilidad:** Al tener la `ingestion_ts` en la PSA, el retail puede demostrar ante una auditoría exactamente cuándo se recibió el dato desde la sucursal, independientemente de cuándo se procesó contablemente.
- **Escalabilidad:** Al usar `WriteToParquet`, Beam creará múltiples archivos pequeños en paralelo, lo que permite que el proceso de lectura posterior sea mucho más rápido al poder leer los archivos de forma distribuida.

Clase 3: Data Warehouse y ETL

Módulo 7: Data Pipelines y plataformas de visualización

Claudio Torres Fonseca

mail: `claudio.torresf@usm.cl`

Universidad de Santiago de Chile

Diplomado en Data Engineer
2026